LegalMind AI Architecture and Evaluation

Solution report | India-focused legal information and debate practice

Abstract

LegalMind AI is a prototype that combines retrieval-augmented legal information with a separate debate-practice agent. It addresses the first-step information gap in routine tenancy, consumer, and wage disputes. The design constrains retrieval by domain, combines semantic and lexical search, preserves source metadata, and uses explicit LangGraph workflows for intake, response generation, and coaching. This report describes implemented components, operating assumptions, evaluation plans, scaling requirements, and known failure modes. The system is not independently validated legal advice.

Keywords: legal RAG; hybrid retrieval; multi-query retrieval; Cohere reranking; LangGraph; LangSmith; India

1. Problem selection and assumptions

We selected everyday disputes such as an unreturned rental deposit, a defective purchase, or unpaid wages because people often need to understand the next practical step before they can reach a lawyer or authority. The product turns a narrative into a structured issue, retrieves legal passages from a bounded corpus, explains possible options, and can prepare a draft or let the user practise an argument. We chose a narrow, evidence-first assistant over an open-domain legal chatbot because source coverage, jurisdiction, and uncertainty are central to legal usefulness.

The current domain lanes are rent and tenancy, consumer protection, and wages. India is the default country context, but state law, forum, contract terms, dates, commencement, and facts may change an answer. The included PDFs are working sources rather than a guarantee of current, complete or authoritative law. The assistant provides information and practice, not representation; consequential filings, deadlines, and notices require qualified local review. The interface is text-based and is not yet multilingual or voice-enabled.

2. Corpus preparation and retrieval

The ingestion pipeline extracts text from legal PDFs, invokes OCR for scanned material, applies conservative cleaning, classifies material by legal domain, and creates structure-aware chunks. Chunks are written to persistent Chroma collections with deterministic IDs and source, page, section, and document metadata. Re-ingestion skips unchanged sources and replaces changed chunks. The BM25 lexical index is reconstructed from persistent chunks on demand and invalidated when collection content changes.

Multi-query retrieval is an optional recall step. A configured small language model produces three alternative phrasings beside the original query, helping bridge ordinary descriptions and statutory language. The implementation rejects duplicate or invalid rewrites and protects factual tokens such as section references, years, dates, amounts, and durations from being dropped. Each query runs domain-scoped retrieval; results are merged and deduplicated with stable chunk IDs.

Hybrid retrieval combines BGE-M3 dense embeddings in Chroma with BM25 keyword search. Dense search can recover conceptually similar passages when wording differs; BM25 helps with exact statutory names, section labels, and phrases. LangChain EnsembleRetriever combines the ranked lists using reciprocal-rank fusion with default equal weights (0.5/0.5), producing up to 20 candidates. The selected legal domain controls the collection; a query does not search across all legal areas.

Cohere is the reranker used by the project configuration. It compares each candidate with the original question, reorders by query-passage relevance, and returns up to six documents by default. Relevance scores and provenance metadata are retained. Reranking reduces noise after recall-oriented retrieval, but it cannot verify a current legal status of a statute, jurisdictional fit, or legal entailment. If no useful passage is retrieved, the answer path should abstain from domain-specific claims.

System workflows, observability, and evaluation

3. Agent architecture and user experience

The normal legal-help StateGraph checks safety, uses a structured supervisor to classify issues, clarifies low-confidence routing, and routes supported domains to retrieval nodes. Drafting precedes citation, urgency, limitation, and section-reference checks; then the conversation node responds.

A separate graph confirms the issue and user position, routes to rent, consumer, or wages, retrieves excerpts, and argues the opposing side. It keeps turn history, switches to coaching on request, and abstains if no relevant excerpt is found.

4. Technology stack and state

Python, LangChain, LangGraph, Chroma, BM25, BGE-M3, Pydantic, and Streamlit form the core stack. Groq or Google chat models are configurable. PyMuPDF, OCRmyPDF, Tesseract, and Pillow support PDFs. PostgreSQL checkpointing is available for durable normal-workflow state.

Streamlit provides Legal Help and Debate Practice modes, new conversations, PDF uploads up to 10 MB, streamed progress, citations, and compliance notices. UI state is in memory and cleared when the process stops; PostgreSQL is optional for the normal graph, while debate defaults to memory.

5. LangSmith observability

Optional LangSmith tracing wraps both streamed graphs when configured with a tracing flag, API key, and optional project or endpoint. Thread metadata helps inspect node paths, latency, and exceptions. Trace content may contain legal narratives or PDF text, so access and retention need controls. Traces diagnose behavior but do not prove accuracy.

6. Evaluation, scaling, and limitations

Repository cases cover exact and semantic retrieval across domains, compare dense, BM25, and hybrid search, and check collection isolation and provenance. The debate case checks setup, routing, source-grounded opposition, continuity, coaching, and abstention. These are plans, not measured results. Next, create expert-labelled source/query sets and report citation entailment, recall, abstention, latency, and cost; add regression and adversarial suites.

Expand through versioned, expert-reviewed central and state sources with effective dates and jurisdiction-aware routing. Release each new domain only after coverage gates. Monitor retrieval quality, unsupported claims, freshness, safety escalations, latency, and cost; restrict sensitive-data access and retention and offer human review.

7. Anti-hallucination design

The domain prompt says retrieved excerpts are the only source for statutory propositions, requires uncertainty, and forbids invented facts, deadlines, sections, jurisdictions, or citations. Intake preserves relative timeline wording and asks for clarification when routing is uncertain; drafting uses stated or explicitly extracted facts and placeholders for missing details. Domain agents assign source IDs from retrieved passages and discard IDs not in that list. The compliance checker flags unmatched IDs or uncited legal information, adds limitation warnings, and flags urgency. The section corrector changes an unsupported section number only when one retrieved section is unambiguous; otherwise it removes the number. Debate prompts require excerpt-grounded claims, and the empty-retrieval path abstains. These controls lower risk but do not test semantic entailment: a valid citation can still fail to support its sentence. A synthetic no-source debate probe returns: "I could not retrieve relevant legal sources in the rent and tenancy collection, so I cannot make a law-based opposing argument yet. You can narrow the issue or share the legal point you want to practise." The appendix records no-source, invalid-citation, and unsupported-section cases with deterministic raw outputs. Live model transcripts were not captured because the provider endpoint was unreachable.

8. Limits and failure modes

Failure modes include OCR or source errors, query drift, reranker misordering, jurisdiction ambiguity, and fluent unsupported claims. Compliance checks citation IDs, deadlines, and urgency but does not establish entailment; limitation rules remain incomplete. Independent legal validation has not been established. Human review is required before consequential action.

Implementation map: agent/prompts/domain.py; agent/nodes/compliance.py; agent/nodes/section_correction.py; agent/workflow.py; frontend/app.py. Test cases and deterministic raw outputs: docs/anti_hallucination_eval.md.